

인스턴스 멤버

- 지금까지 3개의 절에 걸쳐 클래스를 정의하는 방법, 클래스를 기반으로 객체를 인스턴스화하는 방법, 객체의 멤버 데이터와 멤버 함수에 대해 살펴보았음
- 이번 절과 이어지는 다음 절에서는 멤버를 조금 더 자세하게 알아보고, 멤버 사이의 상호 작용에 대해 알아봄
- 클래스를 설계할 때는 인스턴스 멤버와 **클래스 멤버**라는 2가지 멤버 그룹을 사용하므로, 이를 이해하면 클래스를 더 잘 설계할 수 있음

인스턴스 멤버 데이터(1)

- 인스턴스 멤버 데이터(instance data member)는 인스턴스의 속성을 정의
- 따라서 각각의 객체는 클래스에 정의된 멤버 데이터들을 캡슐화해야 함
- 이러한 멤버 데이터는 해당 인스턴스에만 속하므로 인스턴스끼리 서로 접근할 수 없음
- 캡슐화라는 용어는 객체별로 메모리 영역을 할당하므로, 각 영역에서 객체마다 서로 다른 데이터를 갖는 속성을 의미

인스턴스 멤버 데이터(2)

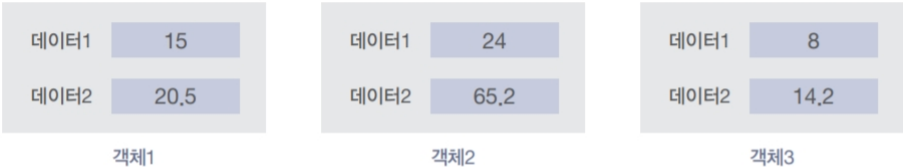


그림 7-9 객체 내부에 캡슐화된 인스턴스 데이터 멤버

인스턴스 멤버 데이터의 접근 제한자(1)

- 인스턴스 멤버 데이터에는 `private`과 `public`이라는 접근 제한자를 적용할 수 있음
- 하지만 인스턴스 멤버 데이터는 일반적으로 `private`으로 설정하는 것이 좋음
- 인스턴스 멤버 데이터를 `public`으로 만들면, 멤버 함수를 호출하지 않고도 애플리케이션에서 멤버 데이터에 접근할 수 있기 때문
- 이는 객체 지향 프로그래밍의 기본적인 개념에 어긋남

인스턴스 멤버 데이터의 접근 제한자(2)

- 객체 지향 프로그래밍은 객체가 행위(멤버 함수)를 통해서 속성을 조작하는 것이 기본
- 따라서 인스턴스 멤버 함수를 통해서만 멤버 데이터에 접근할 수 있도록 인스턴스 멤버 데이터를 `private`으로 만드는 것이 좋음

인스턴스 멤버 함수

- 인스턴스 멤버 함수(instance member function)는 인스턴스의 행위를 의미하며, 객체의 인스턴스 멤버 데이터를 조작하기 위해 사용
- 인스턴스 멤버 데이터는 모든 객체가 개별적으로 갖지만, 멤버 함수는 메모리 위에 하나만 올라가며 모든 인스턴스가 공유



그림 7-10 2개의 데이터 멤버와 4개의 인스턴스 멤버 함수를 갖는 클래스

인스턴스 멤버 함수의 접근 제한자

- 인스턴스 멤버 데이터와 다르게 인스턴스 멤버 함수는 일반적으로 public 접근 제한자를 붙임
- 그래야 클래스 외부(애플리케이션 부분 등)에서 멤버 함수에 접근할 수 있기 때문
- 물론 클래스 내부의 멤버 함수에서만 접근해야 하는 함수를 만들 때도 있음
- 이러한 특별한 경우에는 private 접근 제한자를 붙임

인스턴스 멤버 함수 선택자(1)

- 객체 지향 프로그래밍의 애플리케이션 부분(main 함수 등)에서는 인스턴스를 생성하고, 그 인스턴스가 인스턴스 멤버 함수를 호출하게 해야 함
- 그렇게 해서 인스턴스가 스스로 동작하는 것처럼 코드를 작성하는 것이 객체 지향 프로그래밍의 기본 원리

표 7-4 멤버 선택 연산자

그룹	이름	연산자	표현식	우선 순위	결합방향
후위	멤버 선택	.	객체.멤버	18	→
	멤버 선택	->	포인터->멤버	18	→

인스턴스 멤버 함수 선택자(2)



그림 7-11 멤버 선택 연산자

- 기본적으로 객체마다 멤버함수와 멤버변수를 가지고 있다고 생각하면 맞다.
하지만 컴퓨터 내부적으로는 그렇지 않다.
- 멤버변수의 경우 각 객체의 정보가 보관되는 곳이기 때문에 객체마다 있어야 하지만 멤버함수는 어차피 같은 내용이기 때문에 객체마다 하나씩 있을 필요가 없다.
- 그래서 나온 것이 this 포인터이다.
- 멤버함수는 내부적으로 딱 하나만 존재하지만, 이 멤버함수를 호출하면서 this 포인터를 넘기는 것이다. 그렇게 하면 모든 객체가 마치 자신의 멤버 함수가 있는 것처럼 동작할 수 있다.

this 포인터(2-1)

- 멤버 함수 안에서 자기 자신의 주소를 확인하는 예

```
class WhoAmI
{
public:
    int id;

    WhoAmI(int id_arg);
    void ShowYourself() const;
};

WhoAmI::WhoAmI(int id_arg)
{
    id = id_arg;
}

void WhoAmI::ShowYourself() const
{
    cout << "{ID = " << id << ", this = " << this << "}\n";
}
```

this 포인터(2-2)

- 멤버 함수 안에서 자기 자신의 주소를 확인하는 예

```
int main()
{
    // 세 개의 객체를 만든다.
    WhoAmI obj1( 1);
    WhoAmI obj2( 2);
    WhoAmI obj3( 3);

    // 객체들의 정보를 출력한다.
    obj1.ShowYourself();
    obj2.ShowYourself();
    obj3.ShowYourself();

    // 객체들의 주소를 출력한다.
    cout << "&obj1 = " << &obj1 << "\n";
    cout << "&obj2 = " << &obj2 << "\n";
    cout << "&obj3 = " << &obj3 << "\n";

    return 0;
}
```

this 포인터(2-3)

- 실행 결과

[12-49]

- 멤버 함수의 호출과 가상의 코드

This 포인터(2-4)

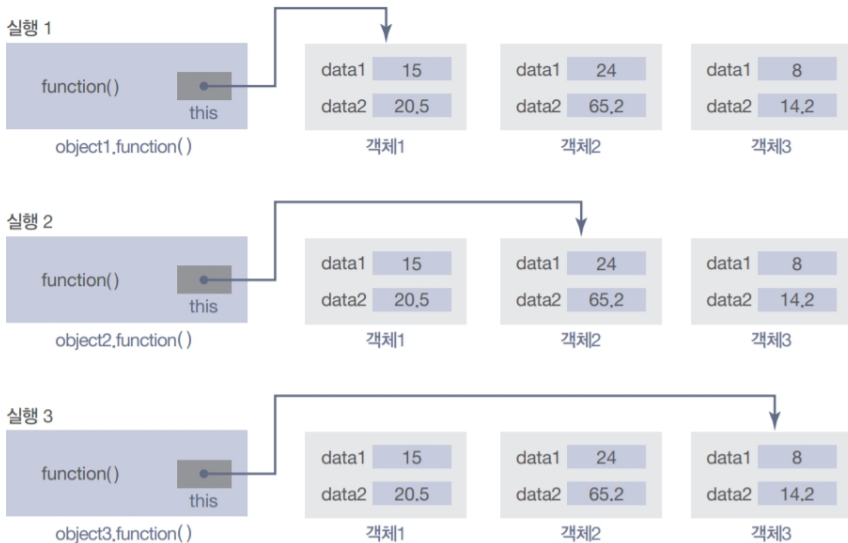


그림 7-12 함수가 특정 객체에 락킹과 언락킹하는 모습

인스턴스 멤버 함수 선택자(6)

- 연산자는 간접 참조 연산자(indirection operator)와 멤버 선택 연산(member operation)이 조합된 특수한 연산자

This 의 data type은 Circle*

this->radius 와
(*this).radius 는
같은 표현식임

인스턴스 멤버 함수 선택자(7)

명시적인 포인터 사용

- 프로그램 내부에서 `this` 포인터를 사용해서 멤버 데이터를 명시적으로 가리킬 수도 있음
- `this` 포인터를 사용하면 멤버 데이터와 매개변수를 같은 이름으로 사용할 수 있음

```
// Without using this pointer
void Circle :: setRadius(double rds)
{
    radius = rds;
}
```

```
// Using this pointer
void Circle :: setRadius(double radius)
{
    this->radius = radius;
}
```

인스턴스 멤버 함수 선택자(8)

호스트 객체

- 호스트 객체(host object)는 인스턴스 멤버 함수를 호출하는 객체를 의미
- 즉 this 포인터가 가리키는 객체가 바로 호스트 객체

접근자 멤버 함수 - getter

- 접근자 멤버 함수(accessor member function)는 호스트 객체의 정보를 추출
get 할 때 사용하는 함수

```
double getRadius () const;           // Host object is read-only
double getPerimeter () const;        // Host object is read-only
double getArea () const;              // Host object is ready-only
```


인스턴스 멤버 함수 선택자(9)

- 접근자 인스턴스 멤버 함수가 객체의 상태를 변경하지 않는다는 것을 확실하게 할 수 있도록 다음과 같이 함수 헤더 끝에 `const` 한정자를 추가하는 것이 좋음
- 접근자 멤버 함수에서 멤버 데이터의 값을 무조건 리턴할 필요는 없으며 다음과 같이 값을 출력하는 기능을 하는 함수도 접근자 멤버 함수라고 할 수 있음

```
void Circle :: output() const
{
    cout << "Radius: " << radius << endl;
    cout << "Perimeter: " << 2 * radius * 3.14 << endl;
    cout << "Area: " << radius * radius * 3.14 << endl;
}
```

인스턴스 멤버 함수 선택자(10)

설정자 멤버 함수

- 멤버 데이터를 변경해야 할 때도 있음
- 이러한 경우에는 호스트 객체의 상태를 변경하는 인스턴스 멤버 함수가 필요하며 이러한 함수가 설정자 멤버 함수(mutator member function)이며 **셋터(setter)**라고 부르기도 함

```
void setRadius(double rds);    // No const qualifier for a
```

mutator

- 설정자 멤버 함수는 호스트 객체의 상태를 변경하므로 const 한정자를 붙이면 안됨
- 설정자 인스턴스 멤버 함수의 역할은 멤버 데이터의 값을 변경하는 것이며 따라서 매개변수가 없어도 멤버 데이터의 값을 변경한다면 설정자 함수

인스턴스 멤버 함수 선택자(11)

- 예를 들어서 다음과 같이 Circle 클래스 객체의 반지름 값을 사용자로부터 입력받아서 멤버 데이터를 변경하는 input 함수도 설정자 멤버 함수

```
void Circle :: input()
{
    cout << "Enter the radius of the circle object: ";
    cin << radius;
}
```

클래스 불변 속성(1)

- 클래스를 디자인할 때 중요한 문제로 클래스 불변 속성이 있음
- 클래스 불변 속성(class invariant)은 클래스의 데이터 멤버의 일부 또는 전체에 적용해야 하는 하나 이상의 조건을 의미하며, 인스턴스 멤버 함수를 사용해서 적용
- 인스턴스를 생성할 때, 생성자에서 수행하는 error checking
- Setter함수에서 수행하는 error checking

클래스 불변 속성(2)

- ‘객체를 생성하는 인스턴스 데이터 멤버 함수(매개변수가 있는 생성자)’ 또는 ‘데이터 멤버의 값을 변경하는 설정자 멤버 함수(setter함수)’를 사용해서 클래스의 불변 속성을 적용

```
Circle :: Circle(double rds)
: radius (rds)
{
    if (radius <= 0.0)
    {
        cout << "No circle can be made!" << endl;
        cout << "The program is aborted" << endl;
        assert(false);
    }
}
```

STATIC MEMBERS

- 이전에 언급했던 것처럼 클래스 자료형에는 인스턴스 멤버와 정적 멤버라는 2가지 멤버가 있음
- 인스턴스 멤버는 살펴보았으므로, 이번 절에서는 정적 멤버를 살펴봄
- 인스턴스 멤버와 마찬가지로 정적 멤버도 정적 멤버 데이터와 정적 멤버 함수로 구분할 수 있음

정적 멤버 (Static Members)

- 정적 멤버란 모든 객체들이 공유하는 멤버를 말한다.

Static Data Member

- 정적 멤버 데이터 static data member 는 클래스 또는 모든 인스턴스에 포함되는 멤버

정적 멤버 데이터 선언

- 정적 멤버 데이터는 클래스 정의에서 선언해야 하고 static이라는 키워드를 붙여야 함
- 다음은 클래스 정의 내부에서 count라는 정적 멤버 데이터를 선언하는 예시

```
class Rectangle
{
    private:
        ...
        static int count;  // Static data member
    public:
        ...
}
```


Static Data Members

정적 멤버 데이터 초기화

- 정적 멤버 데이터는 인스턴스에 속하는 것이 아니므로 생성자에서 초기화할 수 없음
- 정적 멤버 데이터는 클래스 정의 후에 초기화해야 하며 따라서 프로그램의 전역 영역에서 초기화
- 일반적으로는 클래스 정의 바로 뒷부분의 전역 영역에 작성
- 값을 초기화할 때는 클래스 이름과 클래스 스코프 연산자(::)를 추가해서 클래스에 속한다는 것을 나타내야 하며 다만 static 한정자를 추가하면 안됨

```
int Rectangle :: count = 0; // initialization of static data  
member
```

정적 멤버 함수(1)

- 정적 멤버 데이터는 일반적으로 `private`이므로 이에 접근할 수 있는 `public`이 적용된 멤버 함수가 필요
- 인스턴스 멤버 함수에서도 정적 멤버 데이터에 접근할 수 있지만, 일반적으로 정적 멤버 데이터에 접근할 때는 정적 멤버 함수를 사용
- 정적 멤버 함수는 인스턴스와 연결되지 않으므로, 호스트 객체가 없음
- 따라서 정적멤버 함수에서는 인스턴스 멤버 변수에 접근 할 수 없음

정적 멤버 함수 선언

- 정적 멤버 데이터처럼 정적 멤버 함수도 클래스에 속함
- 클래스 내부에 멤버 함수처럼 선언하지만, `static` 키워드를 앞에 붙여야 함

정적 멤버 함수(2)

```
class Rectangle
{
    private:
        ...
        static int count; // Static data member
    public:
        static int getCount(); // Static member function
        ...
}
```

정적 멤버 함수 정의

- 정적 멤버 함수는 인스턴스 멤버 함수처럼 클래스 외부에서 정의
- 정의할 때는 정적 멤버 함수와 인스턴스 멤버 함수의 차이가 없음

```
int Rectangle :: getCount()
{
    return count;
}
```

정적 멤버 함수(3)

정적 멤버 함수 호출

- 정적 멤버 함수는 인스턴스 또는 클래스(Rectangle 등)를 통해서 호출할 수 있음
- 정적 멤버 함수에는 호스트 객체가 없으며 따라서 정적 멤버 함수 내부에서 인스턴스 멤버 데이터에 접근할 수는 없음

```
rect.getCount();           // Through an instance  
Rectangle::getCount();     // Through the class
```

정적 멤버 함수(4)

- 인스턴스 멤버 함수에서는 정적 멤버 데이터에 접근할 수 있음

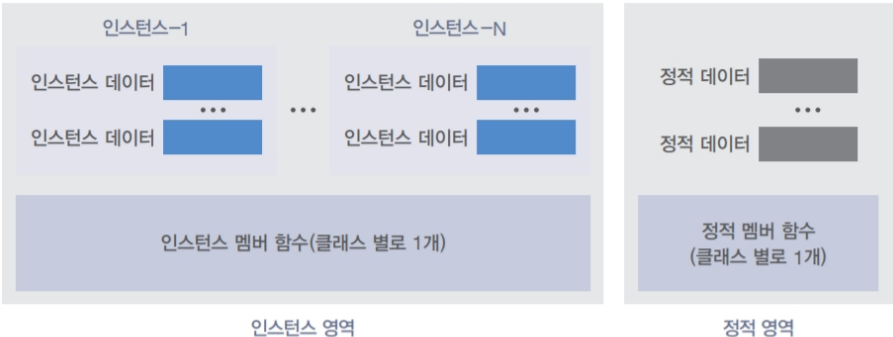


그림 7-13 인스턴스 멤버와 정적 멤버의 영역 분리

정적 멤버를 사용한 객체의 개수 세기(1)

- 정적 멤버를 사용해서 객체의 개수를 세는 예

```
class Student
{
public:
    string name;    // 이름
    int sNo;        // 학번
    Student(const string& name_arg, int stdNumber);
    ~Student();
public:
    // 정적 멤버들
    static int student_count;
    static void PrintStdCount();
};

// 정적 멤버 변수
int Student::student_count = 0;

// 정적 멤버 함수
void Student::PrintStdCount()
{
    cout << "Student 객체 수 = " << student_count << "\n";
}
```

정적 멤버를 사용한 객체의 개수 세기(2)

- 정적 멤버를 사용해서 객체의 개수를 세는 예

```
Student::Student(const string& name_arg, int stdNumber)
{
    // 학생 객체의 수를 증가시킨다.
    student_count++;

    name = name_arg;
    sNo = stdNumber;
}

Student::~~Student()
{
    // 학생 객체의 수를 감소시킨다.
    student_count--;
}
```

정적 멤버를 사용한 객체의 개수 세기(3)

- 정적 멤버를 사용해서 객체의 개수를 세는 예

```
int main()
{
    Student::PrintStdCount();

    Student std("Jeffrey", 123);

    Student::PrintStdCount();

    Func();

    Student::PrintStdCount();

    return 0;
}

void Func()
{
    Student std1("Bill", 342);
    Student std2("James", 214);

    Student::PrintStdCount();
}
```


정적 멤버 정리

- 정적 멤버는 객체의 소유가 아니라 클래스의 소유
- 정적멤버변수/함수, Static멤버변수/함수, 클래스 멤버변수/함수
- 정적멤버변수는 아래와 같이 초기화를 하고 사용
 - 클래스내부에 `static int student_num;`
 - `int Student::student_num=0;`
- 정적멤버 함수에서는 일반 멤버변수를 사용할 수 없다.

정적 멤버 함수(9)

프로그램 7-6 은행 계좌 클래스의 예

```
1  /*****
2  * A program to declare, define, and use a bank account class *
3  *****/
4  #include <iostream>
5  #include <cassert>
6  using namespace std;
7
8  /*****
9  * Class Definition (Declaration of all members          *
10 *****/
11 class Account
12 {
13     private:
14         long accNumber;
15         double balance;
16         static int base; // Static data member
17     public:
18         Account (double bal); // Constructor
19         ~Account (); // Destructor
20         void checkBalance () const; // Accessor
```

정적 멤버 함수(10)

프로그램 7-6 은행 계좌 클래스의 예

```
21         void deposit (double amount); // Mutator
22         void withdraw (double amount); // Mutator
23     };
24     // Initialization of static data member
25     int Account :: base = 0;
26     /*****
27     * Definition of all member functions          *
28     *****/
29     // Parameter Constructor
30     Account :: Account (double bal)
31     :balance (bal)
32     {
33         if (bal < 0.0)
34         {
35             cout << "Balance is negative; program terminates";
36             assert (false);
37         }
38         base++;
39         accNumber = 100000 + base;
40     }
```

정적 멤버 함수(11)

프로그램 7-6 은행 계좌 클래스의 예

```
41     cout << "Account " << accNumber << " is opened. " << endl;
42     cout << "Balance $" << balance << endl << endl;
43 }
44 // Destructor
45 Account :: ~Account ()
46 {
47     cout << "Account #: " << accNumber << " is closed." << endl;
48     cout << "$" << balance << " was sent to the customer." << endl
49     << endl;
50 }
51 // Accessor member function
52 void Account :: checkBalance () const
53 {
54     cout << "Account #: " << accNumber << endl;
55     cout << "Transaction: balance check" << endl;
56     cout << "Balance: $" << balance << endl << endl;
57 }
58 // Mutator Member function
59 void Account :: deposit (double amount)
60 {
61     if (amount > 0.0)
```

정적 멤버 함수(12)

프로그램 7-6 은행 계좌 클래스의 예

```
61     {
62         balance += amount;
63         cout << "Account #: " << accNumber << endl;
64         cout << "Transaction: deposit of $" << amount << endl;
65         cout << "New balance: $" << balance << endl << endl;
66     }
67     else
68     {
69         cout << "Transaction aborted." << endl;
70     }
71 }
72 // Mutator member function
73 void Account::withdraw (double amount)
74 {
75     if (amount > balance)
76     {
77         amount = balance;
78     }
79     balance -= amount;
80     cout << "Account #: " << accNumber << endl;
```

정적 멤버 함수(13)

프로그램 7-6 은행 계좌 클래스의 예

```
81      cout << "Transaction: withdraw of $" << amount << endl;  
82      cout << "New balance: $" << balance << endl << endl;  
83  }  
84  /*****  
85  * Application (the main function) to use the account class      *  
86  *****/  
87  int main ( )  
88  {  
89      // Creation of three accounts  
90      Account acc1 (2000);  
91      Account acc2 (5000);  
92      Account acc3 (1000);  
93      // Transaction  
94      acc1.deposit (150);  
95      acc2.checkBalance ();  
96      acc1.checkBalance ();  
97      acc3.withdraw (800);  
98      acc1.withdraw (1000);  
99      acc2.deposit (120);  
100     return 0;
```

정적 멤버 함수(14)

프로그램 7-6 은행 계좌 클래스의 예

```
101 }
```

Run:

Account 100001 is opened.

Balance \$2000

Account 100002 is opened.

Balance \$5000

Account 100003 is opened.

Balance \$1000

Account #: 100001

Transaction: deposit of \$150

New balance: \$2150

Account #: 100002

Transaction: balance check

Balance: \$5000

정적 멤버 함수(15)

프로그램 7-6 은행 계좌 클래스의 예

```
Account #: 100001  
Transaction: balance check  
Balance: $2150  
  
Account #: 100003  
Transaction: withdraw of $800  
New balance: $200  
  
Account #: 100001  
Transaction: withdraw of $1000  
New balance: $1150  
  
Account #: 100002  
Transaction: deposit of $120  
New balance: $5120  
  
Account #: 100003 is closed.  
$200 was sent to the customer.
```


정적 멤버 함수(16)

프로그램 7-6 은행 계좌 클래스의 예

```
Account #: 100002 is closed.  
$5120 was sent to the customer.
```

```
Account #: 100001 is closed.  
$1150 was sent to the customer.
```

객체 지향 프로그래밍

- 이제 절차 지향 패러다임에서 객체 지향 패러다임으로 한 걸음 더 나아가 보겠음
- 이를 위해서는 프로그램을 컴파일하고 실행하는 방식을 변경해야 함

Separate Files Part 1

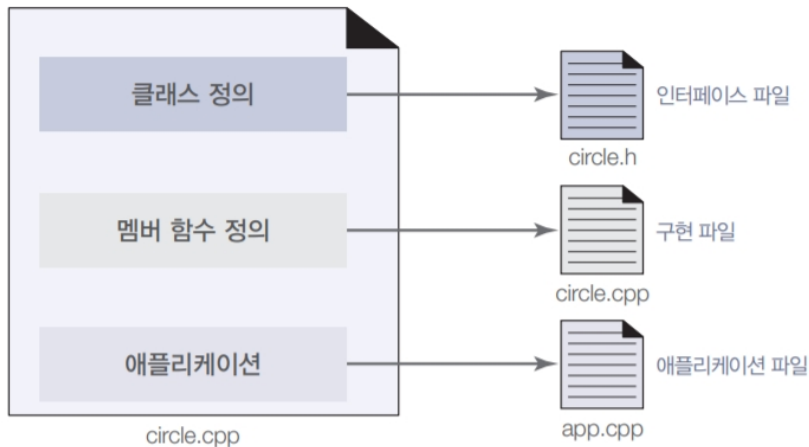


그림 7-14 객체 지향 프로그래밍을 위한 파일 구분

Separate Files Part 2

인터페이스 파일

- 인터페이스 파일은 클래스 정의(멤버 데이터 선언과 멤버 함수 선언)가 포함된 파일
- 이 파일은 클래스의 형태를 다른 파일에 알려주는 역할
- 이 파일의 이름은 일반적으로 circle.h처럼 h 확장자를 붙이며 이때 h라는 문자 이름은 헤더 파일을 의미

구현 파일

- 구현 파일은 멤버 함수 정의가 포함된 파일
- 인터페이스 파일에는 모든 멤버 함수의 선언을 입력
- 이 파일의 이름은 일반적으로 circle.cpp처럼 cpp 확장자를 붙임

Separate Files Part 3

애플리케이션 파일

- 애플리케이션 파일은 객체를 인스턴스화하고 객체를 활용하는 `main` 함수의 코드가 포함된 파일
- 애플리케이션 파일은 반드시 `cpp` 확장자를 사용해야 함
- 파일의 이름은 어떻게 지어도 상관 없지만, 이 책에서는 애플리케이션 파일은 `app.cpp`라는 이름을 붙여서 사용

파일 분리

프로그램 7-7 인터페이스 파일(circle.h)

```
1  /*****
2  * This is the interface file that defines the class Circle.  *
3  * It gives declaration of data members and member functions. *
4  * This file will be included at the top of the implementation *
5  * and application files.                                     *
6  *****/
7  #ifndef CIRCLE_H
8  #define CIRCLE_H
9  #include <iostream>
10 #include <cassert>
11 #include "circle.h"
12 using namespace std;
13 // Class Definition
14 class Circle
15 {
16     private:
17         double radius;
18     public:
19         Circle (double radius); // Parameter constructor
20         Circle (); // Default constructor
```

파일 분리

프로그램 7-7 인터페이스 파일(circle.h)

```
21     Circle (const Circle& circle); // Copy constructor
22     ~Circle (); // Destructor
23     void setRadius (double radius); // Mutator function
24     double getRadius () const; // Accessor function
25     double getArea () const; // Accessor function
26     double getPerimeter () const; // Accessor function
27 };
28 #endif
```

파일 분리

프로그램 7-8 구현 파일(circle.cpp)

```
1  /*****
2  * This is the implementation file that defines the definition *
3  * of all member functions. A copy of the interface file is    *
4  * included at the top to allow compilation of this file.      *
5  *****/
6  #include "circle.h"
7  /*****
8  * The parameter constructor with one argument that initializes *
9  * a circle with the given value. It uses the assert function to *
10 * validate that the radius is a positive double value. If not, *
11 * the program is aborted.                                     *
12 *****/
13 Circle::Circle (double rds)
14 : radius (rds)
15 {
16     if (radius < 0.0)
17     {
18         assert (false);
19     }
20 }
```


파일 분리

프로그램 7-8 구현 파일(circle.cpp)

```
21  /*****
22  * The default constructor that initializes a circle set to 0.0. *
23  * It does not need an assertion.                                *
24  *****/
25  Circle :: Circle ()
26  : radius (0.0)
27  {
28  }
29  /*****
30  * The copy constructor that copies the radius of another circle *
31  * to create a new one. The source circle is already validated, *
32  * which means that we do not need validation.                  *
33  *****/
34  Circle :: Circle (const Circle& circle)
35  : radius (circle.radius)
36  {
37  }
38  /*****
39  * A destructor that cleans up an object when the application is *
40  * terminated.                                                    *
```

파일 분리

프로그램 7-8 구현 파일(circle.cpp)

```
41  *****/
42  Circle :: ~Circle ()
43  {
44  }
45  /******
46  * The setRadius function is defined to change the circle      *
47  * by decreasing or increasing the size of the radius. It needs *
48  * validation because the new size of must be a positive value *
49  *****/
50  void Circle :: setRadius (double value)
51  {
52      radius = value;
53      if (radius < 0.0)
54      {
55          assert (false);
56      }
57  }
58  /******
59  * The getRadius is a function that returns the radius      *
60  * of an object. It needs the const modifier to prevent the  *
```

파일 분리

프로그램 7-8 구현 파일(circle.cpp)

```
61  * accidental change of the host object.                *
62  *****/
63  double Circle :: getRadius () const
64  {
65      return radius;
66  }
67  /******
68  * The getArea accessor function returns the area of the host *
69  * object. It needs the const modifier to prevent the accidental *
70  * change of the host object.                *
71  *****/
72  double Circle :: getArea () const
73  {
74      const double PI = 3.14;
75      return (PI * radius * radius);
76  }
77  /******
78  * The getPerimeter accessor function returns the perimeter of *
79  * the host object. It needs the const modifier to prevent the *
80  * accidental change of the host object.                *
```

파일 분리

프로그램 7-8 구현 파일(circle.cpp)

```
81  *****/
82  double Circle :: getPerimeter () const
83  {
84      const double PI = 3.14;
85      return (2 * PI * radius);
86  }
```

파일 분리

프로그램 7-9 애플리케이션 파일(app.cpp)

```
1  /*****
2  * This is the application file that instantiates objects and      *
3  * lets the object operate on themselves using member functions. *
4  * To be to compiled, it needs a copy of the interface file      *
5  *****/
6  #include "circle.h"
7  int main ( )
8  {
9      // Instantiation of first object and applying operations
10     Circle circle1 (5.2);
11     cout << "Radius: " << circle1.getRadius() << endl;
12     cout << "Area: " << circle1.getArea() << endl;
13     cout << "Perimeter: " << circle1.getPerimeter() << endl;
14     cout << endl;
15     // Instantiation of second object and applying operations
16     Circle circle2 (circle1);
17     cout << "Radius: " << circle2.getRadius() << endl;
18     cout << "Area: " << circle2.getArea() << endl;
19     cout << "Perimeter: " << circle2.getPerimeter() << endl;
20     cout << endl;
```

파일 분리

프로그램 7-9 애플리케이션 파일(app.cpp)

```
21 // Instantiation of third object and applying operations
22 Circle circle3;
23 cout << "Radius: " << circle3.getRadius() << endl;
24 cout << "Area: " << circle3.getArea() << endl;
25 cout << "Perimeter: " << circle3.getPerimeter() << endl;
26 cout << endl;
27 return 0;
28 }
```

파일 분리

프로그램 7-9 애플리케이션 파일(app.cpp)

Run:

Radius: 5.2
Area: 84.9056
Perimeter: 32.656

Radius: 5.2
Area: 84.9056
Perimeter: 32.656

Radius: 0
Area: 0
Perimeter: 0

전처리 지시문

- 같은 헤더 파일을 2회 이상 읽어 들이면 컴파일할 때 오류가 발생하며 이러한 상황을 막으려면 다음과 같이 `define`, `ifndef`, `endif`라는 3가지 전처리 지시문(preprocessor directive)을 사용

```
#ifndef CIRCLE_H
#define CIRCLE_H
    // circle.h 파일의 내용
#endif
```

circle.h

그림 7-16 3가지 전처리 지시문을 추가한 헤더 파일의 형태

전처리 지시문

- `ifndef` 지시문은 `if` 조건문과 비슷
- 만약 플래그가 정의되어 있지 않다면 `ifndef`의 본문을 읽어 들여 `CIRCLE_H`의 코드를 포함
- 플래그가 이미 정의되어 있다면, 이후의 내용을 무시하고 곧바로 `endif` 지시문 위치로 이동

전처리 지시문

```
#ifndef CIRCLE_H
#define CIRCLE_H
    // circle.h 파일의 내용
#endif

...

#ifndef CIRCLE_H
#define CIRCLE_H
    // circle.h 파일의 내용
#endif
```

읽어 들임

무시됨

circle.cpp 또는 app.cpp 파일

그림 7-17 조건부 지시자를 사용해서 중복해서 읽어 들이는 것 막기

클래스 설계

- 설계자가 인터페이스 파일과 구현 파일을 만들며 설계자는 인터페이스 파일만 공개
- 구현 파일의 경우는 컴파일한 것만 공개하고 소스 코드는 비공개로 유지
- 설계자는 필요할 때마다 구현 파일을 변경하고 컴파일한 후에 다시 배포

캡슐화(2)

클래스 사용

- 사용자는 인터페이스 파일의 복사본과 컴파일된 구현 파일을 받음
- 이를 애플리케이션 파일에서 읽어 들이고 컴파일
- 최종적으로 모든 파일을 연결해서 실행 파일을 생성

캡슐화(3)

효과

- 설계자는 사용자의 변경으로부터 인터페이스 파일과 구현 파일을 보호할 수 있음
- 설계자가 컴파일된 구현 파일을 사용자에게 배포하므로, 구현 파일도 사용자가 변경할 수 없음
- 컴파일은 단방향으로 이루어지므로, 컴파일된 구현 파일로는 원본 파일을 구할 수 없음
- 따라서 전체적인 설계는 상자 내부에 캡슐화되어 감추어지며, 사용자가 변경할 수 없다는 의미

캡슐화(4)

공용 인터페이스

- 사용자가 객체를 효율적으로 활용하려면 하나를 더 알아야 함
- 설계자는 일반적으로 공용 인터페이스(public interface)라는 것을 만들어서 제공
- 공용 인터페이스는 사용자가 애플리케이션에서 클래스를 사용하는 방법을 이해할 수 있게 정리한 멤버 함수의 선언과 설명을 정리한 것
- 즉 공용 인터페이스는 함수 정의가 적혀 있는 텍스트 파일로, 사용자에게 클래스의 사용 방법을 설명하는 역할

캡슐화(5)

표 7-5 Circle 클래스의 공용 인터페이스

생성자와 소멸자	역할
Circle::Circle()	원의 반지름을 0.0으로 초기화하는 기본 생성자
Circle::Circle(double radius)	주어진 반지름으로 반지름을 초기화하는 매개변수가 있는 생성자
Circle::Circle(const Circle& circle)	이미 존재하는 원과 같은 원을 만드는 복사 생성자
Circle::~~Circle()	객체를 정리하는 소멸자

접근자 함수	역할
Circle::double getRadius() const	호스트 객체의 반지름(데이터 멤버 radius)을 리턴하는 접근자 함수
Circle::double getArea() const	호스트 객체의 넓이를 리턴하는 접근자 함수
Circle::double getPerimeter() const	호스트 객체의 둘레를 리턴하는 접근자 함수

설정자 함수	역할
Circle::void setRadius(double radius)	호스트 객체의 radius 속성을 변경하는 설정자 함수

클래스 설계

- 분수를 나타내는 `Fraction` 클래스

Fraction 클래스(1)

- 분수는 $3/4$, $1/2$, $7/5$ 처럼 두 정수의 비율을 나타내는
- C++에는 분수를 나타내는 내장 자료형이 없음
- 따라서 분자를 나타내는 값(numer: numerator의 약자)과 분모(denom: denominator의 약자)를 나타내는 값의 int 자료형 2개를 활용해서 새로운 자료형 분수(Fraction)를 만들어야 함

Fraction 클래스(2)

불변 속성

- 새로운 클래스를 만들 때 가장 먼저 고려해야 하는 것은 불변 속성
- 분자와 분모에 약수가 없게 만들며 예를 들어 $6/9$ 은 약분해서 $2/3$ 로 만들어야 함
- 분모는 0일 수 없으며 예를 들어 $2/0$ 를 선언하려 하면 프로그램을 종료
- 분수의 부호는 원래 분자와 분모 부호의 곱셈이며 부호를 분자 쪽에 붙이도록 함
- gcd 함수는 분모와 분자의 최대 공약수를 찾음
- 그리고 normalize 함수는 gcd 함수를 활용해서 위의 불변 속성을 처리

Fraction 클래스(3)

인터페이스 파일

- 이제 fraction.h라는 인터페이스 파일을 생성하며 인터페이스 파일에는 클래스의 정의를 입력

멤버 데이터

- 멤버 데이터로 분모를 나타내는 `denom`과 분자를 나타내는 `numer`를 생성하며 `int` 자료형으로 선언

멤버 함수

- 분수 클래스이므로, 두 분수를 비교하거나 더하거나 하는 여러 가지 멤버 함수를 만들어볼 수 있음

코드

- 프로그램 7-10은 인터페이스 파일

Fraction 클래스(4)

프로그램 7-10 인터페이스 파일(fraction.h)

```
1  /*****
2  * The interface file fraction.h defining the class Fraction  *
3  *****/
4  #include <iostream>
5  using namespace std;
6
7  #ifndef FRACTION_H
8  #define FRACTION_H
9
10 class Fraction
11 {
12     // Data members
13     private:
14         int numer;
15         int denom;
16
17     // Public member functions
18     public:
19         // Constructors
20         Fraction (int num, int den);
```

Fraction 클래스(5)

프로그램 7-10 인터페이스 파일(fraction.h)

```
21     Fraction ();
22     Fraction (const Fraction& fract);
23     ~Fraction ();
24     // Accessors
25     int getNumer () const;
26     int getDenom () const;
27     void print () const;
28     // Mutators
29     void setNumer (int num);
30     void setDenom (int den);
31
32     // Helping private member functions
33     private:
34         void normalize ();
35         int gcd (int n, int m);
36 };
37 #endif
```

Fraction 클래스(6)

구현 파일

- 인터페이스 파일에 정의된 모든 멤버 함수를 정의하는 구현 파일
- 구현 파일에서는 헤더 파일로 인터페이스 파일을 읽어 들임
- 컴파일러가 컴파일 전에 선언을 확인하기 위해서 필요
- `<cassert>` 헤더 파일을 추가했습니다. 이는 분모가 0일 때 프로그램을 중단하고자, `assert` 함수를 활용하기 위해 필요

Fraction 클래스(7)

Implementation File

- ‘분모가 0 일 수 없음’ 조건도 처리
- 분모가 음수일 때는 분자와 분모의 부호를 변경
- 따라서 분모는 양수가 되고, 분자가 음수
- 최대 공약수 함수를 사용해서 분자와 분모의 최대 공약수를 찾은 뒤, 분자와 분모를 나눔
- gcd를 계산할 때는 분자와 분모의 절대값을 사용한다는 것에 주의

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
1  /*****
2  * The implementation file fraction.cpp defining the instance
3  * member functions and helper functions for the Fraction class *
4  *****/
5  #include <iostream>
6  #include <cmath>
7  #include <cassert>
8  #include "fraction.h"
9  using namespace std;
10
11 /*****z
12 * The parameter constructor gets values for the numerator
13 * and denominator, initializes the object, and normalizes the
14 * value of the numerator and the denominator according to the
15 * conditions defined in the class invariant.
16 *****/
17 Fraction :: Fraction (int num, int den = 1)
18 : numer (num), denom (den)
19 {
20     normalize ();
```


Fraction 클래스(9)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
21 }
22 /*****
23 * The default constructor creates a fraction as 0/1. It does *
24 * not need validation. *
25 *****/
26 Fraction :: Fraction ( )
27 : numer (0), denom (1)
28 {
29 }
30 /*****
31 * The copy constructor creates a new fraction from an existing *
32 * object. It does not need normalization because the source *
33 * object is already normalized. *
34 *****/
35 Fraction :: Fraction (const Fraction& fract )
36 : numer (fract.numer), denom (fract.denom)
37 {
38 }
39 /*****
40 * The destructor simply cleans up a fraction for recycling. *
```

Fraction 클래스(10)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
41  *****/
42  Fraction :: ~Fraction ()
43  {
44  }
45  /******
46  * The getNumer function is an accessor function returning the *
47  * numerator of the host object. It needs the const modifier *
48  *****/
49  int Fraction :: getNumer () const
50  {
51      return numer;
52  }
53  /******
54  * The getDenom function is an accessor function returns the *
55  * denominator of the host object. It needs the const modifier *
56  *****/
57  int Fraction :: getDenom () const
58  {
59      return denom;
60  }
```

Fraction 클래스(11)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
61  /*****
62  * The print function is an accessor function with a side effect *
63  * that display the fraction object in the form x/y.           *
64  *****/
65  void Fraction :: print () const
66  {
67      cout << numer << "/" << denom << endl;
68  }
69  /*****
70  * The setNumer is a mutator function the changes the numerator *
71  * of an existing object. The object needs normalization.      *
72  *****/
73  void Fraction :: setNumer (int num)
74  {
75      numer = num;
76      normalize();
77  }
78  /*****
79  * The setDenom is a mutator function that changes denominator *
80  * of an existing object. The object needs normalization.      *
```

Fraction 클래스(12)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
81  *****/
82  void Fraction :: setDenom (int den)
83  {
84      denom = den;
85      normalize();
86  }
87  /**/
88  * Normalize function takes care of three fraction invariants. *
89  *****/
90  void Fraction :: normalize ()
91  {
92      // Handling a denominator of zero
93      if (denom == 0)
94      {
95          cout << "Invalid denomination. Need to quit." << endl;
96          assert (false);
97      }
98      // Changing the sign of denominator
99      if (denom < 0)
100     {
```

Fraction 클래스(13)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
101         denom = - denom;
102         numer = - numer;
103     }
104     // Dividing numerator and denominator by gcd
105     int divisor = gcd (abs(numer), abs (denom));
106     numer = numer / divisor;
107     denom = denom / divisor;
108 }
109 /*****
110 * The gcd function finds the greatest common divisor between *
111 * the numerator and the denominator. *
112 *****/
113 int Fraction :: gcd (int n, int m)
114 {
115     int gcd = 1;
116     for (int k = 1; k <= n && k <= m; k++)
117     {
118         if (n % k == 0 && m % k == 0)
119         {
120             gcd = k;
```

Fraction 클래스(14)

프로그램 7-11 모든 멤버 함수를 정의하는 파일(fraction.cpp)

```
121     }  
122     }  
123     return gcd;  
124 }
```

애플리케이션 파일

애플리케이션 파일은 사용자가 만드는 프로그램

프로그램 7-12 애플리케이션 파일(app.cpp)

```
1  /*****
2  * The application file app.cpp uses the Fraction objects.      *
3  *****/
4  #include "fraction.h"
5  #include <iostream>
6  using namespace std;
7
8  int main ( )
9  {
10     // Instantiation of some objects
11     Fraction fract1 ;
12     Fraction fract2 (14, 21);
13     Fraction fract3 (11, -8);
14     Fraction fract4 (fract3);
15     // Printing the object
16     cout << "Printing four fractions after constructed: " << endl;
17     cout << "fract1: ";
18     fract1. print();
19     cout << "fract2: ";
20     fract2. print();
```

Fraction 클래스(16)

프로그램 7-12 애플리케이션 파일(app.cpp)

```
21     cout << "fract3: ";
22     fract3.print();
23     cout << "fract4: ";
24     fract4.print();
25     // Using mutators
26     cout << "Changing the first two fractions and printing them:" <<
27     endl;
28     fract1.setNumer(4);
29     cout << "fract1: ";
30     fract1.print();
31     fract2.setDenom(-5);
32     cout << "fract2: ";
33     fract2.print();
34     // Using accessors
35     cout << "Testing the changes in two fractions:" << endl;
36     cout << "fract1 numerator: " << fract1.getNumer() << endl;
37     cout << "fract2 numerator: " << fract2.getDenom() << endl;
38     return 0;
```


Fraction 클래스(17)

프로그램 7-12 애플리케이션 파일(app.cpp)

```
c++ -c fraction.cpp           // Compilation of implementation
file
c++ -c app.cpp                // Compilation of application file
c++ -o application fraction.o app.o // Linking of two compiled object
files
```

Run:

Printing four fractions after constructed:

fract1: 0/1

fract2: 2/3

fract3: -11/8

fract4: -11/8

Changing the first two fractions and printing them:

fract1: 4/1

fract2:-2/5

Testing the changes in two fractions:

Numerator of fract1: 4

Denominator of fract2: 5

화떡 이야기:

상효는 정문에서 인스턴스

태운이는 가천관에서 인스턴스

누구는 공학관에서 인스턴스

영미는 CEO야

전체 매출이 궁금해 클래스멤버

Home work (static must)

- You operate several hot dog stands distributed throughout town. Define a class named HotDogStand that has a member variable for the hot dog stand's ID number and a member variable for how many hot dogs the stand sold that day.
- Create a constructor that allows a user of the class to initialize both values.
- Also create a function named JustSold() that increments the number of hot dogs so that you can track the total number of hot dogs sold by the stand.
- Add another function that returns the number of hot dogs sold.
- Finally, add a static variable that tracks the total number of hot dogs sold by all hot dog stands and a static function that returns the value in this variable.
- Write a main function to test your class with at least three hot dog stands that each sell a variety of hot dogs.

Home work (static must)

- ```
main() {
 int count;
 HotDogStand* sList;
 cout << "Stand count(37~107) : ";
 cin >> count;
 sList = new HotDogStand[count];
 for (int i = 0; i < count; i++) {
 sList[i].SetID(i);
 }
 ...
}
```

## Home work (static must)

- 실행예제

Stand count(3개~10개) : 3

ID : 0

ID : 0

ID : 2

ID : 2

ID : 2

ID : 1

ID : q